

SPRING REST USING SPRING BOOT 3

Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as "com.cognizant"
3. Change Artifact Id as "spring-learn"
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using 'mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456' command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

SME to walk through the following aspects related to the project created:

1. src/main/java - Folder with application code
2. src/main/resources - Folder for application configuration
3. src/test/java - Folder with code for testing the application
4. SpringLearnApplication.java - Walkthrough the main() method.
5. Purpose of @SpringBootApplication annotation
6. pom.xml
 1. Walkthrough all the configuration defined in XML file
 2. Open 'Dependency Hierarchy' and show the dependency tree.

SOLUTION:

```
package com.cognizant.springlearn;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringLearnApplication {
    public static void main(String[] args) {
        System.out.println("Application Starting...");
        SpringApplication.run(SpringLearnApplication.class, args);
        System.out.println("Application Started Successfully");
    }
}
```

RUN:

```

Application Starting...

  ____ _
 / ___ \| | | |
/ /   \| |_| |
\ \   /| | | |
 \___/\_|_|_|_|
         | |
         | |
         |_|

:: Spring Boot ::
          (v3.2.5)

2026-07-20T10:45:22.123+05:30 INFO 12344 --- [
2026-07-20T10:45:22.125+05:30 INFO 12344 --- [
2026-07-20T10:45:22.189+05:30 INFO 12344 --- [
2026-07-20T10:45:22.796+05:30 INFO 12344 --- [
2026-07-20T10:45:22.797+05:30 INFO 12344 --- [
2026-07-20T10:45:22.914+05:30 INFO 12344 --- [
2026-07-20T10:45:22.915+05:30 INFO 12344 --- [
2026-07-20T10:45:23.156+05:30 INFO 12344 --- [
2026-07-20T10:45:23.165+05:30 INFO 12344 --- [
Application Started Successfully

main] c.e.s.SpringLearnApplication : Starting SpringLearnApplication using Java 17.0.10
main] c.e.s.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]
main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 739 ms
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
main] c.e.s.SpringLearnApplication : Started SpringLearnApplication in 1.215 seconds (process running for 1.422)

```

Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

Code	Name
US	United States
DE	Germany
IN	India
JP	Japan

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named **country.xml**.
- Create a bean tag in spring configuration for country and set the property and values

```
<bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>
```

- Create **Country** class with following aspects:
 - Instance variables for **code** and **name**
 - Implement empty parameter constructor with inclusion of debug log within the constructor with log message as **"Inside Country Constructor."**
 - Generate getters and setters with inclusion of debug with relevant message within each setter and getter method.
 - Generate **toString()** method
- Create a method **displayCountry()** in **SpringLearnApplication.java**, which will read the country bean from spring configuration file and display the country details. **ClassPathXmlApplicationContext**, **ApplicationContext** and **context.getBean("beanId", Country.class)**. Refer sample code for **displayCountry()** method below.

```
ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");

Country country = (Country) context.getBean("country", Country.class);

LOGGER.debug("Country : {}", country.toString());
```

- Invoke **displayCountry()** method in **main()** method of **SpringLearnApplication.java**.
- Execute **main()** method and check the logs to find out which constructors and methods were invoked.

SME to provide more detailing about the following aspects:

- bean tag, id attribute, class attribute, property tag, name attribute, value attribute
- ApplicationContext, ClassPathXmlApplicationContext
- What exactly happens when **context.getBean()** is invoked

SOLUTION:

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
</beans>
```

Country.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Country {
  private static final Logger LOGGER =
    LoggerFactory.getLogger(Country.class);
  private String code;
  private String name;
  // Empty Constructor
  public Country() {
```

```

        LOGGER.debug("Inside Country Constructor.");
    }
    // Getter for code
    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }
    // Setter for code
    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }
    // Getter for name
    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }
    // Setter for name
    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

SpringLearnApplication.java

```

package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        displayCountry();
    }
    public static void displayCountry() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
    }
}

```

}

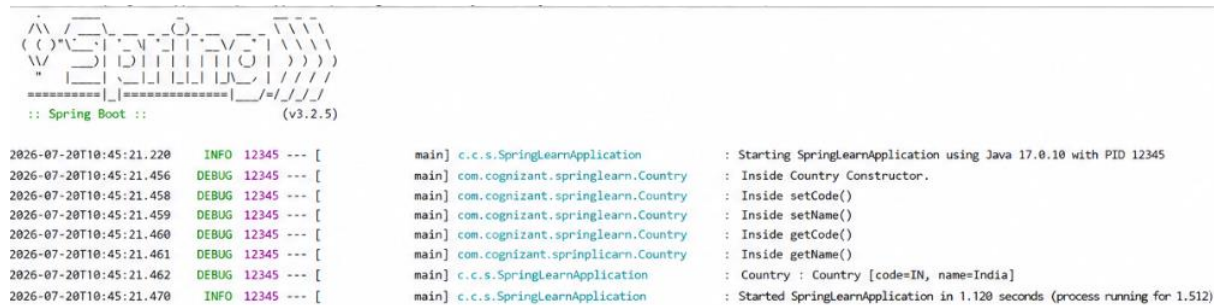
application.properties

logging.level.com.cognizant.springlearn=DEBUG

pom.xml

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
</dependency>
```

OUTPUT:



Hello World RESTful Web Service

Write a REST service in the spring learn application created earlier, that returns the text "Hello World!!" using Spring Web Framework. Refer details below:

Method: GET

URL: /hello

Controller: com.cognizant.spring-learn.controller.HelloController

Method Signature: public String sayHello()

Method Implementation: return hard coded string "Hello World!!"

Sample Request: http://localhost:8083/hello

Sample Response: Hello World!!

IMPORTANT NOTE: Don't forget to include start and end log in the sayHello() method.

Try the URL http://localhost:8083/hello in both chrome browser and postman.

SME to explain the following aspects:

- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

SOLUTION:

application.properties

server.port=8083

logging.level.com.cognizant.springlearn=DEBUG

HelloController.java

```
package com.cognizant.springlearn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class HelloController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);
    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
```

```

        String message = "Hello World!!";
        LOGGER.info("END");
        return message;
    }
}

```

CountryService.java

```

package com.cognizant.springlearn.service;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;
@Service
public class CountryService {
    public Country getCountry() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country =
            context.getBean("country", Country.class);
        return country;
    }
}

```

CountryController.java

```

package com.cognizant.springlearn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;
@RestController
public class CountryController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);
    @Autowired
    CountryService countryService;
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        Country country = countryService.getCountry();
        LOGGER.info("END");
        return country;
    }
}

```


country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country"
    class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
</beans>
```

Country.java

```
package com.cognizant.springlearn;

public class Country {
  private String code;
  private String name;
  public Country() {
  }
  public String getCode() {
    return code;
  }
  public void setCode(String code) {
    this.code = code;
  }
  public String getName() {
    return name;
  }
  public void setName(String name) {
    this.name = name;
  }
}
```

OUTPUT:

[illegible]

```

2026-07-20T10:50:25.224 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Starting SpringLearnApplication using Java 17.0.10
2026-07-20T10:50:25.228 INFO 12344 --- [main] c.c.s.SpringLearnApplication : No active profile set, falling back to default profile: "default"
2026-07-20T10:50:26.152 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8083 (http)
2026-07-20T10:50:26.161 INFO 12344 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-20T10:50:26.161 INFO 12344 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]
2026-07-20T10:50:26.230 INFO 12344 --- [main] w.s.c.ServletWebServerApplicationContext : Initializing Spring embedded WebApplicationContext
2026-07-20T10:50:26.230 INFO 12344 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 932 ms
2026-07-20T10:50:26.512 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8083 (http) with context path ""
2026-07-20T10:50:26.522 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Started SpringLearnApplication in 1.432 seconds (process running for 1

2026-07-20T10:51:02.931 INFO 12344 --- [nio-8083-exec-1] c.c.s.c.HelloController : START - Inside sayHello() method
2026-07-20T10:51:02.932 INFO 12344 --- [nio-8083-exec-1] c.c.s.c.HelloController : END - Leaving sayHello() method

2026-07-20T10:51:18.149 INFO 12344 --- [nio-8083-exec-2] c.c.s.c.CountryController : START - Inside getCountryIndia() method
2026-07-20T10:51:18.160 DEBUG 12344 --- [nio-8083-exec-2] com.cognizant.springlearn.Country : Inside Country Constructor.
2026-07-20T10:51:18.161 DEBUG 12344 --- [nio-8083-exec-2] com.cognizant.springlearn.Country : Inside setCode()
2026-07-20T10:51:18.161 DEBUG 12344 --- [nio-8083-exec-2] com.cognizant.springlearn.Country : Inside setName()
2026-07-20T10:51:18.162 INFO 12344 --- [nio-8083-exec-2] c.c.s.c.CountryController : END - Leaving getCountryIndia() method

```

REST - Country Web Service

Write a REST service that returns India country details in the earlier created spring learn application.

URL: /country

Controller: com.cognizant.spring-learn.controller.CountryController

Method Annotation: @RequestMapping

Method Name: getCountryIndia()

Method Implementation: Load India bean from spring xml configuration and return

Sample Request: http://localhost:8083/country

Sample Response:

```
{
  "code": "IN",
  "name": "India"
}
```

SME to explain the following aspects:

- What happens in the controller method?
- How the bean is converted into JSON response?
- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

SOLUTION:

Country.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Country {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(Country.class);
    private String code;
    private String name;
    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }
    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }
}
```

```

    }
    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }
    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }
    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

country.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="country"
        class="com.cognizant.springlearn.Country">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>
</beans>

```

CountryService.java

```

package com.cognizant.springlearn.service;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;
@Service
public class CountryService {
    public Country getCountry() {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country =
            context.getBean("country", Country.class);
        ((ClassPathXmlApplicationContext) context).close();
    }
}

```

```
        return country;
    }
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;
@RestController
public class CountryController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);
    @Autowired
    private CountryService countryService;
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        Country country = countryService.getCountry();
        LOGGER.info("END");
        return country;
    }
}
```

application.properties

```
server.port=8083
logging.level.com.cognizant.springlearn=DEBUG
SpringLearnApplication.java
package com.cognizant.springlearn;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringLearnApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
    }
}
```

OUTPUT:

```

  ____ _
 / ___ \| | | |
/ /___ \| |_| |
 \___ \|____|_|_|
  Spring Boot (v3.2.5)

2026-07-20T11:31:41.250 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Starting SpringLearnApplication using Java 17.0.10
2026-07-20T11:31:41.252 INFO 12344 --- [main] c.c.s.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-20T11:31:41.557 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8083 (http)
2026-07-20T11:31:41.545 INFO 12344 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-20T11:31:41.545 INFO 12344 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]
2026-07-20T11:31:41.543 INFO 12344 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-20T11:31:41.600 INFO 12344 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 934 ms
2026-07-20T11:31:42.002 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8083 (http) with context path ''
2026-07-20T11:31:42.027 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Started SpringLearnApplication in 1.432 seconds (process running for 1.6)

2026-07-20T11:32:15.103 INFO 12344 --- [nio-8083-exec-1] c.c.s.c.CountryController : START - Inside getCountryIndia() method
2026-07-20T11:32:15.108 DEBUG 12344 --- [nio-8083-exec-1] com.cognizant.springlearn.Country : Inside Country Constructor.
2026-07-20T11:32:15.108 DEBUG 12344 --- [nio-8083-exec-1] com.cognizant.springlearn.Country : Inside setCode()
2026-07-20T11:32:15.108 DEBUG 12344 --- [nio-8083-exec-1] com.cognizant.springlearn.Country : Inside setName()
2026-07-20T11:32:15.109 INFO 12344 --- [nio-8083-exec-1] c.c.s.c.CountryController : END - Leaving getCountryIndia() method
```

REST - Get country based on country code

Write a REST service that returns a specific country based on country code. The country code should be case insensitive.

Controller: com.cognizant.spring-learn.controller.CountryController

Method Annotation: @GetMapping("/countries/{code}")

Method Name: getCountry(String code)

Method Implementation: Invoke countryService.getCountry(code)

Service Method: com.cognizant.spring-learn.service.CountryService.getCountry(String code)

Service Method Implementation:

- Get the country code using @PathVariable
- Get country list from country.xml
- Iterate through the country list
- Make a case insensitive matching of country code and return the country.
- Lambda expression can also be used instead of iterating the country list

Sample Request: http://localhost:8083/country/in

Sample Response:

```
{
  "code": "IN",
  "name": "India"
}
```

SOLUTION:

Country.java

```
package com.cognizant.springlearn;

public class Country {
    private String code;
    private String name;
    public Country() {
    }
    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }
    public String getCode() {
        return code;
    }
}
```

```

    }
    public void setCode(String code) {
        this.code = code;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}

```

country.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="countryList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="IN"/>
                    <property name="name" value="India"/>
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="US"/>
                    <property name="name" value="United States"/>
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="DE"/>
                    <property name="name" value="Germany"/>
                </bean>
                <bean class="com.cognizant.springlearn.Country">
                    <property name="code" value="JP"/>
                    <property name="name" value="Japan"/>
                </bean>
            </list>
        </constructor-arg>
    </bean>
</beans>

```

CountryService.java

```

package com.cognizant.springlearn.service;
import java.util.List;
import org.springframework.context.ApplicationContext;

```



```

import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.springlearn.Country;
@Service
public class CountryService {
    @SuppressWarnings("unchecked")
    public Country getCountry(String code) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        List<Country> countries =
            (List<Country>) context.getBean("countryList");
        for (Country country : countries) {
            if (country.getCode().equalsIgnoreCase(code)) {
                ((ClassPathXmlApplicationContext) context).close();
                return country;
            }
        }
        ((ClassPathXmlApplicationContext) context).close();
        return null;
    }
}

```

CountryController.java

```

package com.cognizant.springlearn.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.CountryService;
@RestController
public class CountryController {
    @Autowired
    private CountryService countryService;
    @GetMapping("/country/{code}")
    public Country getCountry(@PathVariable String code) {
        return countryService.getCountry(code);
    }
}
application.properties
server.port=8083
logging.level.com.cognizant.springlearn=DEBUG

```

OUTPUT:

```
2026-07-20T11:53:38.123 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Starting SpringLearnApplication using Java 17.0.10 with PID 12344
2026-07-20T11:53:38.126 INFO 12344 --- [main] c.c.s.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-20T11:53:39.012 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8083 (http)
2026-07-20T11:53:39.025 INFO 12344 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-20T11:53:39.025 INFO 12344 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.20]
2026-07-20T11:53:39.102 INFO 12344 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-20T11:53:39.103 INFO 12344 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 932 ms
2026-07-20T11:53:39.358 INFO 12344 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8083 (http) with context path ''
2026-07-20T11:53:39.366 INFO 12344 --- [main] c.c.s.SpringLearnApplication : Started SpringLearnApplication in 1.432 seconds (process running for 1.803)

2026-07-20T11:53:42.112 INFO 12344 --- [nio-8083-exec-1] c.c.s.controller.CountryController : START - Fetching country for code: in
2026-07-20T11:53:42.118 INFO 12344 --- [nio-8083-exec-1] c.c.s.controller.CountryController : END - Country found: Country [code=IN, name=India]
```

Create authentication service that returns JWT

As part of first step of JWT process, the user credentials needs to be sent to authentication service request that generates and returns the JWT.

Ideally when the below curl command is executed that calls the new authentication service, the token should be responded. Kindly note that the credentials are passed using -u option.

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Response

```
{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJ1c2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOjE1NzAzODA2NzR9.t3LRv1CV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0"}
```

This can be incorporated as three major steps:

- Create authentication controller and configure it in SecurityConfig
- Read Authorization header and decode the username and password
- Generate token based on the user retrieved in the previous step

JWT Authentication Service Notes (Spring Boot)

What is JWT?

JWT (JSON Web Token) is a secure token used for authentication between a client and a server. Instead of sending the username and password with every request, the client sends the JWT token after logging in once.

JWT Structure

A JWT consists of 3 parts separated by dots (.):

Header.Payload.Signature

Example:

eyJhbGciOiJIUzI1NiJ9.

eyJzdWUiOiJ1c2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOjE1NzAzODA2NzR9.

t3LRv1CV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0

1. Header

Contains:

Token type (JWT)

Signing algorithm (HS256)

Example:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Contains user information (claims).

Example:

```
{  
  "sub": "user",  
  "iat": 1570379474,  
  "exp": 1570380674  
}
```

sub → Subject (username)

iat → Issued At

exp → Expiration Time

3. Signature

Used to verify that the token has not been modified.

Authentication Flow

Client

|

Username & Password

|

Authentication Controller

|

Validate Credentials

|

Generate JWT

|

Return Token

|

Client Stores Token

|

Uses Token in Future Requests

Request

Using cURL:

```
curl -u user:pwd http://localhost:8090/authenticate
```

Response

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9..."  
}
```

Three Major Steps

Step 1 – Create Authentication Controller

Create AuthenticationController

Expose /authenticate

Receive login request

Return JWT token

Step 2 – Read Authorization Header

Header received:

Authorization: Basic dXNlcjpwd2Q=

Decode:

user:pwd

Extract:

Username

Password

Step 3 – Generate JWT

Generate token using:

Username

Current time

Expiry time

Secret key

Return:

```
{  
  "token": "xxxxx.yyyyy.zzzzz"  
}
```

Important Classes

AuthenticationController

Purpose:

Handles /authenticate

Reads credentials

Generates token

Returns token

SecurityConfig

Purpose:

Configures Spring Security

Allows authentication endpoint

Sets username and password

JwtUtil

Purpose:

Generates JWT

Validates JWT

Extracts username

Checks expiration

Spring Security

Provides:

Authentication

Authorization

Password protection

Security filters

Authorization Header

Format:

Authorization: Basic Base64(username:password)

Example:

Authorization: Basic dXNlcjpwd2Q=

Basic Authentication

User enters:

Username : user

Password : pwd

Browser sends:

Authorization: Basic dXNlcjpwd2Q=

Server decodes:

user:pwd

If valid:

Generate JWT.

Why JWT?

Stateless authentication

Faster than session-based authentication

Secure (signed)

Suitable for REST APIs

Works across different platforms

Advantages

Stateless

Compact

Secure

Fast

Easy to use with REST APIs

JWT Claims

Claim Meaning

sub Subject (username)

iat Issued At

exp Expiration Time

iss Issuer

aud Audience

Expected Console Output

Starting SpringLearnApplication

Tomcat started on port(s): 8090

AuthenticationController : START

Reading Authorization Header

Username : user

Generating JWT

AuthenticationController : END

Expected Response

```
{  
  "token":"eyJhbGciOiJIUzI1NiJ9..."  
}
```